CAN Protocol

Engineering Handbook

STM32F4 • ESP32 • S32K1 • AUTOSAR • KV5x

Full register-level reference with complete bit definitions

Version 2 — Expanded Register Definitions

1. CAN Protocol Fundamentals

1.1 Overview

Controller Area Network (CAN) is a robust, multi-master serial communication protocol originally developed by Bosch in the 1980s for automotive applications. It is now standardized under ISO 11898 and widely used in automotive, industrial automation, medical, and aerospace systems. CAN uses differential signaling (CAN_H and CAN_L) on a two-wire bus, providing excellent noise immunity and the ability to detect and handle errors without a bus master.

Key characteristics of CAN:

- Multi-master broadcast bus — any node can transmit when the bus is idle.
- Message-based (not address-based) — each frame carries an identifier indicating message priority and type.
- Non-destructive bitwise arbitration — higher-priority messages win access to the bus without data loss.
- Extensive error detection — CRC, bit monitoring, bit stuffing, frame check, ACK check.
- Fault confinement — nodes automatically reduce activity when error rates exceed thresholds.

1.2 CAN Bus Physical Layer

CAN uses differential signaling over a twisted pair. The two states are:

Bus State	CAN_H Voltage	CAN_L Voltage	Differential (H-L)	Logic
Recessive (logic 1)	2.5 V (floating)	2.5 V (floating)	~0 V	1
Dominant (logic 0)	3.5 V	1.5 V	~2.0 V	0

The bus is terminated at each end with a 120 Ω resistor to match the characteristic impedance of the cable. The maximum bus length depends on the bit rate:

Bit Rate	Max Bus Length	Max Propagation Delay
1 Mbit/s	25 m	5 ns/m (typical)
500 kbit/s	100 m	
250 kbit/s	250 m	
125 kbit/s	500 m	
50 kbit/s	1000 m	
10 kbit/s	5000 m	

1.3 CAN Frame Types

CAN defines four frame types:

Frame Type	Purpose	Key Fields
Data Frame	Carries data from transmitter to receivers	SOF, Arbitration ID, RTR=0, DLC, Data (0-8 bytes), CRC, ACK, EOF
Remote Frame	Requests data from another node	Same as Data Frame but RTR=1, no data payload
Error Frame	Signals a detected error on the bus	Error Flag (6-12 dominant bits) + Error Delimiter (8 recessive bits)
Overload Frame	Injects delay between data/remote frames	Overload Flag + Overload Delimiter

1.4 Standard (CAN 2.0A) vs Extended (CAN 2.0B) Frame Format

Field	Standard Frame (CAN 2.0A)	Extended Frame (CAN 2.0B)
Identifier	11 bits (2048 IDs)	29 bits (536M IDs)
RTR/SRR	RTR bit	SRR=1 (substitute), RTR after IDE
IDE bit	0 (dominant)	1 (recessive)
Total overhead	~47 bits (0 data bytes)	~67 bits (0 data bytes)
Max payload	8 bytes	8 bytes

1.5 CAN FD (Flexible Data-Rate)

CAN FD (ISO 11898-1:2015) extends classic CAN with two key improvements:

- Payload up to 64 bytes (vs 8 bytes in classic CAN).
- Two-phase bit timing: arbitration phase at nominal bit rate (same as classic CAN); data phase at a higher bit rate (up to 8–10 Mbit/s with proper transceivers).

CAN FD frame additions:

Field	Classic CAN	CAN FD
FDF (FD Frame) bit	Not present	1 = CAN FD frame
BRS (Bit Rate Switch)	Not present	1 = switch to data bit rate
ESI (Error State Indicator)	Not present	Reflects transmitter error state
DLC	0–8 (linear)	0–15 (non-linear: 0-8, 12, 16, 20, 24, 32, 48, 64)
CRC	15-bit	17-bit (≤16 bytes) or 21-bit (>16 bytes)

Stuff Bit Count	Not present	Fixed 4-bit gray-coded stuff counter

CAN FD requires FD-capable transceivers (e.g., TJA1044, TCAN1042) and all nodes on the bus must support CAN FD if FD frames are transmitted.

1.6 Bit Timing and Synchronization

Each CAN bit is divided into time quanta (TQ). The bit time consists of four segments:

- Sync Segment (always 1 TQ) — used to synchronize nodes on edge detection.
- Propagation Segment (1–8 TQ) — compensates for physical bus delay.
- Phase Buffer Segment 1 / Phase Seg 1 (1–8 TQ) — compensates for edge phase error, can be lengthened.
- Phase Buffer Segment 2 / Phase Seg 2 (2–8 TQ) — compensates for edge phase error, can be shortened.

The sample point is at the end of Phase Seg 1. Typical sample point: 75%–87.5% of bit time. The Synchronization Jump Width (SJW, 1–4 TQ) defines the maximum adjustment per resynchronization.

Bit rate formula:

```
Bit Rate = CAN Clock / (Prescaler * (1 + Tseg1 + Tseg2))

Example: CAN clock = 48 MHz, Prescaler=6, Tseg1=13, Tseg2=2
  TQ = 6/48MHz = 125 ns
  Bit time = (1 + 13 + 2) * 125 ns = 16 * 125 ns = 2000 ns
  Bit rate = 1/2000 ns = 500 kbit/s
  Sample point = (1+13)/16 * 100% = 87.5%
```

1.7 CAN Error Handling

Every CAN node maintains two error counters: Transmit Error Counter (TEC) and Receive Error Counter (REC). Based on these counters, a node operates in one of three states:

State	TEC / REC Threshold	Behavior
Error Active	TEC < 128 and REC < 128	Normal operation. Sends Active Error Flag (6 dominant bits).
Error Passive	TEC ≥ 128 or REC ≥ 128	Sends Passive Error Flag (6 recessive bits). Must wait 8 recessive bits before retransmitting.
Bus Off	TEC ≥ 256	Node disconnects from bus. Recovery requires 128 occurrences of 11 consecutive recessive bits.

Bus Off recovery is automatic in many hardware implementations but can be configured as manual (software-triggered) for safety-critical systems.

2. CAN on STM32F4 (bxCAN)

2.1 Hardware Overview

STM32F4 bxCAN supports CAN 2.0A/B with 3 TX mailboxes, two 3-message RX FIFOs, 28 filter banks (shared between CAN1/CAN2), TTCAN, auto-retransmit, and auto wake-up. Does NOT support CAN FD.

2.2 Complete Register Bit Definitions

CAN_MCR — Master Control Register

Bit(s)	Name	Reset	Description
16	`DBF`	0	Debug freeze. 0: CAN works normally in debug mode; 1: CAN reception/transmission frozen when debug mode is active (SRAM accessible).
15	`RESET`	0	bxCAN software master reset. Write 1 to reset all registers and state machines to reset values. Self-clearing.
7	`TTCM`	0	Time-triggered communication mode. 0: disabled; 1: time-stamp of each received/transmitted message captured from internal 16-bit free-running counter.
6	`ABOM`	0	Automatic bus-off management. 0: software must initiate recovery by requesting/leaving initialisation mode; 1: hardware automatically exits bus-off after 128 × 11 recessive bits.
5	`AWUM`	0	Automatic wake-up mode. 0: software must clear SLEEP bit to wake up; 1: hardware exits sleep on detection of CAN message activity.
4	`NART`	0	No automatic retransmission. 0: hardware retransmits automatically on arbitration loss or error; 1: each message attempted only once regardless of result.
3	`RFLM`	0	Receive FIFO locked mode. 0: FIFO overrun — oldest message overwritten; 1: FIFO locked — new message discarded when FIFO full.
2	`TXFP`	0	Transmit FIFO priority. 0: priority driven by message identifier (lower ID = higher priority); 1: priority driven by order of transmit requests (chronological FIFO).
1	`SLEEP`	1	Sleep mode request. 1: CAN in sleep mode (low power, clock gated); 0: normal mode. Cleared by software or auto-wake-up.
0	`INRQ`	0	Initialisation request. Write 1 to enter initialisation mode (required to configure BTR and filters). Hardware confirms with INAK=1 in MSR.

CAN_MSR — Master Status Register

Bit(s)	Name	Reset	Description
11	`RX`	—	CAN Rx signal (read-only). Current logic level of the CAN_RX pin — useful for bus monitoring.
10	`SAMP`	—	Last sample point value (read-only). Logic level of the CAN bus at the last sample point.
9	`RXM`	—	Receive mode (read-only). 1: CAN is currently receiving a message.
8	`TXM`	—	Transmit mode (read-only). 1: CAN is currently transmitting a message.
4	`SLAKI`	0	Sleep acknowledge interrupt. Set when SLAK=1 (sleep mode entered). Cleared by writing 0. Generates interrupt if SLKIE=1.
3	`WKUI`	0	Wake-up interrupt. Set by hardware when a SOF is detected while in sleep mode. Write 0 to clear. Generates interrupt if WKUIE=1.
2	`ERRI`	0	Error interrupt. Set when an error condition is detected (see ESR). Write 0 to clear. Generates interrupt if ERRIE=1.
1	`SLAK`	—	Sleep acknowledge (read-only). 1: CAN is in sleep mode. Mirrors SLEEP request once hardware has acknowledged it.
0	`INAK`	—	Initialisation acknowledge (read-only). 1: CAN hardware is in initialisation mode (confirms INRQ=1 request was accepted).

CAN_BTR — Bit Timing Register (only writable when INRQ=1)

Bit(s)	Name	Reset	Description
31	`SILM`	0	Silent mode (debug). 0: normal; 1: CAN in silent mode — receives normally but does not transmit (monitors bus without participating).
30	`LBKM`	0	Loop back mode (debug). 0: normal; 1: CAN in loop-back — transmitter connected internally to receiver. Useful for self-test without physical bus.
25:24	`SJW[1:0]`	0	Resynchronisation jump width minus 1. Maximum number of TQ the bit time can be lengthened or shortened during resynchronisation. Range: 1–4 TQ.
22:20	`TS2[2:0]`	0	Time segment 2 minus 1 (Phase Seg 2). Duration after sample point. Range: 1–8 TQ. Must be ≥ SJW.
19:16	`TS1[3:0]`	0	Time segment 1 minus 1 (Prop Seg + Phase Seg 1). Duration from sync segment end to sample point. Range: 1–16 TQ.
9:0	`BRP[9:0]`	0	Baud rate prescaler minus 1. TQ period = (BRP+1) / APB1_clock. Range: 1–1024.

CAN_TSR — Transmit Status Register

Bit(s)	Name	Reset	Description
31	`LOW2`	—	Mailbox 2 lowest priority flag. Set if mailbox 2 lost arbitration against mailboxes 0/1.
30	`LOW1`	—	Mailbox 1 lowest priority flag.
29	`LOW0`	—	Mailbox 0 lowest priority flag.
28	`TME2`	—	Transmit mailbox 2 empty (read-only). 1: mailbox 2 is empty and available.
27	`TME1`	—	Transmit mailbox 1 empty (read-only).
26	`TME0`	—	Transmit mailbox 0 empty (read-only). Poll TME0/1/2 to find a free mailbox before writing.
25:24	`CODE[1:0]`	—	Mailbox code (read-only). Indicates which mailbox is next to be transmitted: 00=MB0, 01=MB1, 10=MB2. Set by hardware based on priority.
19	`ABRQ2`	0	Abort request mailbox 2. Write 1 to request abort of pending transmission in MB2. Hardware clears after abort completes.
18	`TXOK2`	—	Transmission OK mailbox 2. Set by hardware when MB2 transmission succeeds. Write 0 to clear.
17	`ALST2`	—	Arbitration lost mailbox 2. Set when MB2 lost arbitration. Write 0 to clear.
16	`TERR2`	—	Transmission error mailbox 2. Set when MB2 transmission failed due to error. Write 0 to clear.
15	`RQCP2`	0	Request completed mailbox 2. Set when last request (tx or abort) in MB2 is complete. Write 0 to clear and re-arm.
12	`ABRQ1`	0	Abort request mailbox 1.
9	`ABRQ0`	0	Abort request mailbox 0.

CAN_RF0R / CAN_RF1R — Receive FIFO 0 / 1 Register

Bit(s)	Name	Reset	Description
5	`RFOM0/1`	0	Release FIFO output mailbox. Write 1 to release (consume) the oldest message from the FIFO. Hardware clears after release. Must write after reading each message.
4	`FOVR0/1`	0	FIFO overrun. Set when FIFO is full and a new message arrives. Cleared by writing 0. Indicates missed message.
3	`FULL0/1`	0	FIFO full flag. Set when FIFO contains 3 messages (maximum). Cleared by hardware when a message is released.

1:0	`FMP[1:0]`	0	FIFO message pending (read-only). Number of messages currently stored in FIFO: 00=0, 01=1, 10=2, 11=3. Generates FMPIE interrupt when > 0.

CAN_IER — Interrupt Enable Register

Bit(s)	Name	Reset	Description
17	`SLKIE`	0	Sleep interrupt enable. 1: interrupt when sleep mode is entered (SLAKI=1).
16	`WKUIE`	0	Wake-up interrupt enable. 1: interrupt when wake-up event detected (WKUI=1).
15	`ERRIE`	0	Error interrupt enable. 1: interrupt when any error flag in ESR is set.
11	`LECIE`	0	Last error code interrupt enable. 1: interrupt when LEC[2:0] in ESR is updated.
10	`BOFIE`	0	Bus-off interrupt enable. 1: interrupt when BOFF flag is set in ESR.
9	`EPVIE`	0	Error passive interrupt enable. 1: interrupt when EPVF is set (error passive state entered/left).
8	`EWGIE`	0	Error warning interrupt enable. 1: interrupt when EWGF is set (TEC or REC ≥ 96).
6	`FOVIE1`	0	FIFO 1 overrun interrupt enable.
5	`FFIE1`	0	FIFO 1 full interrupt enable.
4	`FMPIE1`	0	FIFO 1 message pending interrupt enable.
3	`FOVIE0`	0	FIFO 0 overrun interrupt enable.
2	`FFIE0`	0	FIFO 0 full interrupt enable.
1	`FMPIE0`	0	FIFO 0 message pending interrupt enable. 1: interrupt when FMP0 > 0 (message available in FIFO 0).
0	`TMEIE`	0	Transmit mailbox empty interrupt enable. 1: interrupt when any TX mailbox becomes empty (TME0/1/2 set).

CAN_ESR — Error Status Register

Bit(s)	Name	Reset	Description
31:24	`REC[7:0]`	0	Receive error counter (read-only). Incremented on receive errors, decremented on successful reception. ≥128 → error passive state.
23:16	`TEC[7:0]`	0	Transmit error counter (read-only). Incremented on transmit errors, decremented on successful transmission. ≥128 → error passive; ≥256 → bus-off.

6:4	`LEC[2:0]`	0	Last error code (read-only, updated each error). 000=no error; 001=stuff error; 010=form error; 011=acknowledgement error; 100=bit recessive error; 101=bit dominant error; 110=CRC error; 111=set by software.
2	`BOFF`	0	Bus-off flag (read-only). 1: CAN is in bus-off state (TEC exceeded 255).
1	`EPVF`	0	Error passive flag (read-only). 1: CAN is in error passive state (TEC or REC ≥ 128).
0	`EWGF`	0	Error warning flag (read-only). 1: TEC or REC has reached the error warning limit of 96.

CAN_TIxR — TX Mailbox Identifier Register (x = 0, 1, 2)

Bit(s)	Name	Reset	Description
31:21	`STID[10:0]`	0	Standard identifier (11-bit). Bits [31:21] hold the 11-bit CAN ID when IDE=0. Also holds EID[28:18] part of extended ID when IDE=1.
20:3	`EXID[17:0]`	0	Extended identifier (18 LSBs). Used when IDE=1 to form the full 29-bit extended ID: STID[10:0] | EXID[17:0].
2	`IDE`	0	Identifier extension. 0: standard 11-bit ID; 1: extended 29-bit ID.
1	`RTR`	0	Remote transmission request. 0: data frame; 1: remote frame (requests data, no payload).
0	`TXRQ`	0	Transmit mailbox request. Write 1 to request transmission of this mailbox. Hardware clears after transmission is complete.

CAN_TDTxR — TX Mailbox Data Length and Time Stamp Register

Bit(s)	Name	Reset	Description
31:16	`TIME[15:0]`	0	Message time stamp. Captured from the CAN free-running timer when TTCM=1. Written to this field for TTCAN time-triggered transmission.
8	`TGT`	0	Transmit global time. When TTCM=1 and TGT=1, the current 16-bit timer value is sent in the last two bytes of the data field automatically.
3:0	`DLC[3:0]`	0	Data length code. Number of bytes in the data field: 0–8 for classic CAN. Values 9–15 are valid but treated as 8 bytes by most hardware.

CAN_FMR — Filter Master Register

Bit(s)	Name	Reset	Description

13:8	`CAN2SB[5:0]`	0x0E	CAN2 start bank. Defines which filter bank (0–27) is the first assigned to CAN2. Banks 0..(CAN2SB-1) belong to CAN1; CAN2SB..27 belong to CAN2.
0	`FINIT`	0	Filter initialisation mode. 1: filter registers can be written (filters deactivated); 0: filters active. Must set FINIT=1 before changing filter config, clear after.

CAN_FA1R — Filter Activation Register

Bit(s)	Name	Reset	Description
27:0	`FACT[27:0]`	0	Filter active bits. One bit per filter bank (0–27). 0: filter bank disabled (all messages pass or blocked depending on config); 1: filter bank active. Only writable when FINIT=1.

CAN_FM1R — Filter Mode Register

Bit(s)	Name	Reset	Description
27:0	`FBM[27:0]`	0	Filter bank mode. One bit per bank. 0: mask mode — one ID + one mask, accepts range of IDs; 1: list mode — two exact IDs (32-bit) or four exact IDs (16-bit). Only writable when FINIT=1.

CAN_FS1R — Filter Scale Register

Bit(s)	Name	Reset	Description
27:0	`FSC[27:0]`	0	Filter scale. One bit per bank. 0: dual 16-bit filter (two ID/mask pairs or four IDs); 1: single 32-bit filter (one ID/mask pair or two IDs). Only writable when FINIT=1.

CAN_FFA1R — Filter FIFO Assignment Register

Bit(s)	Name	Reset	Description
27:0	`FFA[27:0]`	0	Filter FIFO assignment. One bit per bank. 0: matching messages go to FIFO0; 1: matching messages go to FIFO1. Only writable when FINIT=1.

2.3 HAL Usage Example

```
CAN_HandleTypeDef hcan1;
void MX_CAN1_Init(void){
    hcan1.Instance=CAN1;
    hcan1.Init.Prescaler=6;
    hcan1.Init.Mode=CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth=CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1=CAN_BS1_11TQ;
    hcan1.Init.TimeSeg2=CAN_BS2_2TQ;
    hcan1.Init.AutoBusOff=ENABLE;
    hcan1.Init.AutoRetransmission=ENABLE;
    HAL_CAN_Init(&hcan1);
```

```
}
/* Transmit */
CAN_TxHeaderTypeDef hdr={.StdId=0x123,.IDE=CAN_ID_STD,.RTR=CAN_RTR_DATA,.DLC=8};
uint8_t data[8]={1,2,3,4,5,6,7,8}; uint32_t mb;
HAL_CAN_AddTxMessage(&hcan1,&hdr,data,&mb);
/* Receive */
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *h){
    CAN_RxHeaderTypeDef rxHdr; uint8_t rxData[8];
    HAL_CAN_GetRxMessage(h,CAN_RX_FIFO0,&rxHdr,rxData);
}
```

3. CAN on ESP32 (TWAI Controller)

3.1 Hardware Overview

ESP32 TWAI (Two-Wire Automotive Interface) is compatible with CAN 2.0B (ISO 11898-1). NOT CAN FD capable. One TX buffer, 64-byte RX FIFO, acceptance filtering, three modes (Normal, Self-Test, Listen-Only), configurable alerts. External transceiver required (e.g., SN65HVD230, TJA1050).

3.2 TWAI Hardware Registers — Full Bit Definitions

TWAI registers are mapped in the I/O peripheral address space. They use the SJA1000 register format for compatibility.

TWAI_MODE_REG — Mode Register (offset 0x000)

Bit(s)	Name	Reset	Description
4	`AFM`	0	Acceptance filter mode. 0: dual filter mode (two 16-bit filters); 1: single filter mode (one 32-bit filter). Must be set while in reset mode.
3	`STM`	0	Self-test mode. 0: normal (ACK required from another node); 1: self-test (node ACKs its own transmission — no other node needed).
2	`LOM`	0	Listen-only mode. 0: normal operation; 1: node listens but does not transmit ACK or error frames. Bus is not disturbed.
1	`RM`	1	Reset mode. 1: controller in reset — registers configurable, no bus activity; 0: operating mode. Always enter reset mode before changing configuration.

TWAI_CMD_REG — Command Register (offset 0x004) — write-only

Bit(s)	Name	Reset	Description
4	`SRR`	0	Self reception request. Write 1 to transmit and simultaneously receive back the same message (self-test without LOM).
3	`CDO`	0	Clear data overrun. Write 1 to clear the data overrun status flag in SR. Must clear before FIFO is re-enabled.
2	`RRB`	0	Release receive buffer. Write 1 to release (pop) the oldest frame from the RX FIFO and advance to the next. Must write after reading each received frame.
1	`AT`	0	Abort transmission. Write 1 to cancel a pending transmission request. Useful to abort a frame before it is sent.
0	`TR`	0	Transmission request. Write 1 to begin transmission of the frame loaded into TX buffer. Cleared by hardware when transmission starts.

TWAI_STATUS_REG — Status Register (offset 0x008) — read-only

Bit(s)	Name	Reset	Description
7	`BS`	0	Bus status. 0: bus-on (normal); 1: bus-off (TEC exceeded 255). Node has disconnected from bus and is waiting for recovery.
6	`ES`	0	Error status. 0: error active (TEC and REC < 128); 1: error passive (TEC or REC ≥ 128). Reflects transmitter error state.
5	`TS`	0	Transmit status. 0: idle; 1: currently transmitting a frame.
4	`RS`	0	Receive status. 0: idle; 1: currently receiving a frame (between SOF and EOF).
3	`TCS`	1	Transmission complete status. 1: last requested transmission successfully completed; 0: last requested transmission has not yet completed.
2	`TBS`	1	Transmit buffer status. 1: TX buffer free and available for a new frame; 0: TX buffer occupied (transmission pending or ongoing).
1	`DOS`	0	Data overrun status. 1: a message was lost because the RX FIFO was full when it arrived. Clear using CDO command.
0	`RBS`	0	Receive buffer status. 1: one or more frames available in RX FIFO; 0: RX FIFO empty. Clear by issuing RRB command after reading.

TWAI_INT_REG — Interrupt Register (offset 0x00C) — read-only, cleared on read

Bit(s)	Name	Reset	Description
7	`BEI`	0	Bus error interrupt. Set when a bit error, stuff error, CRC error, form error, or ACK error is detected during a frame.
6	`ALI`	0	Arbitration lost interrupt. Set when the node loses bus arbitration (another node's dominant bit overwrites a recessive bit during TX).
5	`EPI`	0	Error passive interrupt. Set when the controller changes between error passive and error active states (threshold crossing at TEC/REC=128).
4	`WUI`	0	Wake-up interrupt. Set when the controller wakes up from sleep mode after detecting bus activity.
3	`DOI`	0	Data overrun interrupt. Set when a received frame is discarded because the RX FIFO is full (data overrun occurred).
2	`EI`	0	Error warning interrupt. Set when TEC or REC reaches or drops below the error warning limit (typically 96).
1	`TI`	0	Transmit interrupt. Set when a frame has been successfully transmitted and the TX buffer is released.

0	`RI`	0	Receive interrupt. Set when a frame is successfully received and stored in the RX FIFO.

TWAI_INT_ENA_REG — Interrupt Enable Register (offset 0x010)

Bit(s)	Name	Reset	Description
7	`BEIE`	0	Bus error interrupt enable. 1: generate interrupt when BEI is set.
6	`ALIE`	0	Arbitration lost interrupt enable. 1: generate interrupt when ALI is set.
5	`EPIE`	0	Error passive interrupt enable. 1: generate interrupt when EPI is set.
4	`WUIE`	0	Wake-up interrupt enable.
3	`DOIE`	0	Data overrun interrupt enable.
2	`EIE`	0	Error warning interrupt enable.
1	`TIE`	0	Transmit interrupt enable.
0	`RIE`	0	Receive interrupt enable.

TWAI_BUS_TIMING_0_REG — Bus Timing Register 0 (offset 0x018)

Bit(s)	Name	Reset	Description
7:6	`SJW[1:0]`	0	Synchronisation jump width minus 1. 00=1 TQ, 01=2 TQ, 10=3 TQ, 11=4 TQ. Maximum phase error correction per bit.
5:0	`BRP[5:0]`	0	Baud rate prescaler. TQ = 2 × (BRP+1) / f_APB. E.g., BRP=4 at 80 MHz → TQ = 125 ns.

TWAI_BUS_TIMING_1_REG — Bus Timing Register 1 (offset 0x01C)

Bit(s)	Name	Reset	Description
7	`SAM`	0	Sampling mode. 0: single sampling at sample point; 1: three samples taken, majority vote used. Use 1 for noisy environments below 125 kbit/s.
6:4	`TSEG2[2:0]`	0	Time segment 2 minus 1 (Phase Seg 2). TQ after sample point. Range 1–8 TQ. Must be ≥ SJW.
3:0	`TSEG1[3:0]`	0	Time segment 1 minus 1 (Prop Seg + Phase Seg 1). TQ before sample point. Range 1–16 TQ.

TWAI_ALC_REG — Arbitration Lost Capture Register (offset 0x02C) — read-only

Bit(s)	Name	Reset	Description

4:0	**ALC[4:0]**	—	Arbitration lost capture. Bit position (0–28) at which arbitration was lost during the last transmission. Bit 0 = first bit of identifier. Useful for diagnosing bus contention.

TWAI_ECC_REG — Error Code Capture Register (offset 0x030) — read-only

Bit(s)	Name	Reset	Description
7:6	**ERRC[1:0]**	—	Error code. 00=bit error; 01=form error; 10=stuff error; 11=other error.
5	**TXWRN**	—	TX error warning. 1: error occurred during transmission.
4	**RXWRN**	—	RX error warning. 1: error occurred during reception.
3:0	**SEG[3:0]**	—	Segment code. Indicates which part of the CAN frame the error occurred in: 0x03=SOF; 0x02=ID.28-21; 0x06=ID.20-18; 0x04=SRTR; 0x05=IDE; 0x07=ID.17-13; 0x0F=ID.12-5; 0x0E=ID.4-0; 0x0C=RTR; 0x0D=res.1; 0x09=res.0; 0x0B=DLC; 0x0A=data; 0x08=CRC seq.; 0x18=CRC delim.; 0x19=ACK slot; 0x1B=ACK delim.; 0x1A=EOF; 0x12=intermission.

TWAI_ERR_WARNING_LIMIT_REG — Error Warning Limit Register (offset 0x034)

Bit(s)	Name	Reset	Description
7:0	**EWL[7:0]**	0x60	Error warning limit (default 96 = 0x60). When TEC or REC reaches this value, the error warning flag (ES bit 6 in SR, EI interrupt) is set. Configurable; must be in reset mode to change.

TWAI_RX_ERR_CNT_REG / TWAI_TX_ERR_CNT_REG (offsets 0x038 / 0x03C)

Bit(s)	Name	Reset	Description
7:0	**RXB_ERR_CNT[7:0]**	0	Receive error counter (read-only normally; writable in reset mode for testing). Incremented on RX errors, decremented on successful reception.
7:0	**TXB_ERR_CNT[7:0]**	0	Transmit error counter (writable in reset mode). Incremented on TX errors (×8 for some error types), decremented on successful transmission.

3.3 ESP-IDF API Example

```
#include "driver/twai.h"
void twai_init(void){
    twai_general_config_t g=TWAI_GENERAL_CONFIG_DEFAULT(21,22,TWAI_MODE_NORMAL);
    g.alerts_enabled=TWAI_ALERT_BUS_ERROR|TWAI_ALERT_BUS_OFF;
    twai_timing_config_t t=TWAI_TIMING_CONFIG_500KBITS();
    twai_filter_config_t f=TWAI_FILTER_CONFIG_ACCEPT_ALL();
    twai_driver_install(&g,&t,&f);
    twai_start();
}
```

```
void twai_send(uint32_t id,uint8_t *data,uint8_t len){
    twai_message_t m={.identifier=id,.extd=0,.rtr=0,.data_length_code=len};
    memcpy(m.data,data,len);
    twai_transmit(&m,pdMS_TO_TICKS(100));
}
```

4. CAN on S32K1 (FlexCAN)

4.1 Hardware Overview

S32K1xx FlexCAN supports CAN 2.0A/B and CAN FD (on K14x/K148). Features 32–64 configurable Message Buffers, Rx FIFO, up to 128 individual/global masks, listen-only mode, loopback, TTCAN, DMA (K14x+), and TDC (transmitter delay compensation for CAN FD).

4.2 Complete Register Bit Definitions

CAN_MCR — Module Configuration Register

Bit(s)	Name	Reset	Description
31	`MDIS`	0	Module disable. 0: FlexCAN enabled; 1: module clock gated off. Set MDIS=0 to enable; must be enabled before entering freeze mode.
30	`FRZ`	0	Freeze enable. When FRZ=1 and HALT=1 (or debug event), CAN enters freeze mode — registers become writable. Must be 1 to configure CAN after first enable.
29	`RFEN`	0	Rx FIFO enable. 0: Rx FIFO disabled; 1: Rx FIFO enabled. When enabled, the first 8 MBs (0–7) are reserved for FIFO use.
28	`HALT`	0	Halt request. Write 1 to request freeze/halt. Confirmed when FRZACK=1 in MCR. Write 0 to resume operation.
27	`NOTRDY`	—	FlexCAN not ready (read-only). 1: module initialising or in low-power mode; 0: ready for transmission/reception.
26	`WAKMSK`	0	Wake-up interrupt mask. 0: wake-up interrupt disabled; 1: interrupt generated on self-wake-up event.
25	`SOFTRST`	0	Soft reset. Write 1 to reset internal state and FIFOs (not configuration registers). Self-clearing after reset completes.
24	`FRZACK`	—	Freeze mode acknowledge (read-only). 1: FlexCAN is in freeze mode and register writes are permitted.
22	`SUPV`	1	Supervisor mode. 0: user mode — all registers accessible; 1: supervisor mode — some registers only accessible in supervisor privilege level.
21	`SLFWAK`	0	Self wake-up. 0: normal sleep; 1: FlexCAN can wake from stop/doze mode on CAN bus activity.
20	`WRNEN`	0	Warning interrupt enable. 0: Tx/Rx warning interrupts disabled; 1: TWRNINT/RWRNINT bits in ESR1 generate interrupts.
19	`DOZE`	0	Doze mode enable. 0: enabled in doze; 1: clock gated in doze mode.

17	`LPDIS`	0	Low-power mode disable. 0: FlexCAN can enter low-power mode on STOP request; 1: prevented from entering low-power mode.
16	`IRMQ`	0	Individual Rx masking and queue enable. 0: global mask applies to all MBs; 1: RXIMR[n] array provides per-MB individual masks.
14:13	`IDAM[1:0]`	0	ID acceptance mode (Rx FIFO). 00: one full 32-bit ID per filter element; 01: two 16-bit IDs per element; 10: four 8-bit partial IDs; 11: all frames accepted (mask disabled).
12	`FDEN`	0	CAN FD operation enable. 0: classic CAN; 1: CAN FD frames enabled. S32K14x/K148 only.
6:0	`MAXMB[6:0]`	0x0F	Maximum number of message buffers. Defines how many MBs (0 to MAXMB) are active. Must be set before enabling CAN (HALT=0).

CAN_CTRL1 — Control 1 Register

Bit(s)	Name	Reset	Description
31:24	`PRESDIV[7:0]`	0	Prescaler division factor. CAN clock = peripheral clock / (PRESDIV+1). Determines TQ period.
23:22	`RJW[1:0]`	0	Resync jump width minus 1. Maximum phase error correction: 1–4 TQ.
21:19	`PSEG1[2:0]`	0	Phase segment 1 minus 1. Phase Seg 1 duration: 1–8 TQ. Lengthened during resynchronisation.
18:16	`PSEG2[2:0]`	0	Phase segment 2 minus 1. Phase Seg 2 duration: 2–8 TQ (minimum 2 to ensure propagation). Shortened during resync.
15	`BOFFMSK`	0	Bus off interrupt mask. 0: disabled; 1: interrupt when BOFFINT is set in ESR1.
14	`ERRMSK`	0	Error interrupt mask. 0: disabled; 1: interrupt when ERRINT is set in ESR1.
13	`CLKSRC`	0	CAN engine clock source. 0: oscillator clock; 1: peripheral/bus clock. Must match timing calculations.
12	`LPB`	0	Loop-back mode enable. 0: normal; 1: TX internally connected to RX for self-test. Does not affect physical bus.
11	`TWRNMSK`	0	TX warning interrupt mask. 0: disabled; 1: interrupt when TWRNINT=1 in ESR1 (TEC ≥ TWRN threshold).
10	`RWRNMSK`	0	RX warning interrupt mask. 0: disabled; 1: interrupt when RWRNINT=1 in ESR1 (REC ≥ RWRN threshold).
7	`SMP`	0	CAN bit sampling mode. 0: one sample per bit; 1: three samples, majority vote. Use 1 for noisy environments at low speeds.

6	`BOFFREC`	0	Bus off recovery mode. 0: automatic recovery after 128×11 recessive bits; 1: recovery blocked until software requests it.
5	`TSYN`	0	Timer sync mode. 1: CAN free-running timer resets on reception of SOF. Enables bus-wide time synchronisation.
4	`LBUF`	0	Lowest buffer transmitted first. 0: lowest ID (highest priority) MB transmitted first; 1: lowest MB number transmitted first.
3	`LOM`	0	Listen-only mode. 0: normal operation; 1: receive only — no ACK, no error frames sent. Bus is not disturbed.
2:0	`PROPSEG[2:0]`	0	Propagation segment minus 1. Compensates for bus propagation delay. Range 1–8 TQ. Bit time = sync(1) + PROPSEG + PSEG1 + PSEG2 (in TQ).

CAN_ESR1 — Error and Status 1 Register

Bit(s)	Name	Reset	Description
19	`SYNCH`	—	CAN synchronisation status (read-only). 1: CAN is synchronised to the bus and can transmit/receive.
18	`TWRNINT`	0	TX warning interrupt flag. Set when TEC reaches the warning threshold. Write 1 to clear.
17	`RWRNINT`	0	RX warning interrupt flag. Set when REC reaches the warning threshold. Write 1 to clear.
16	`BIT1ERR`	0	Bit 1 error. Detected recessive but monitored dominant during transmission. Write 1 to clear.
15	`BIT0ERR`	0	Bit 0 error. Detected dominant but monitored recessive during TX. Write 1 to clear.
14	`ACKERR`	0	ACK error. No dominant bit received in ACK slot (no node acknowledged the frame). Write 1 to clear.
13	`CRCERR`	0	CRC error. Received CRC did not match calculated CRC. Write 1 to clear.
12	`FRMERR`	0	Form error. Fixed-format field (SOF, EOF, ACK delimiter, CRC delimiter) contained illegal bit. Write 1 to clear.
11	`STFERR`	0	Stuffing error. More than 5 consecutive bits of the same polarity detected (violates bit stuffing rule). Write 1 to clear.
10	`TXWRN`	—	TX error counter warning (read-only). 1: TEC ≥ 96 (error warning threshold).
9	`RXWRN`	—	RX error counter warning (read-only). 1: REC ≥ 96.
8	`IDLE`	—	CAN bus idle (read-only). 1: 11 consecutive recessive bits detected — bus is idle.
7	`TX`	—	FlexCAN is transmitting (read-only). 1: TX in progress.

6:5	`FLTCONF[1:0]`	—	Fault confinement state (read-only). 00: error active; 01: error passive; 1x: bus off.
4	`RX`	—	FlexCAN is receiving (read-only). 1: RX in progress.
2	`BOFFINT`	0	Bus off interrupt flag. Set when FlexCAN enters bus-off state. Write 1 to clear.
1	`ERRINT`	0	Error interrupt flag. Set when any error condition occurs. Write 1 to clear.
0	`WAKINT`	0	Wake-up interrupt flag. Set when a wake-up event is detected. Write 1 to clear.

CAN_IMASK1 — Interrupt Mask 1 Register

Bit(s)	Name	Reset	Description
31:0	`BUF31M–BUF0M`	0	Message buffer interrupt mask. One bit per MB (bit 0=MB0, bit 31=MB31). 0: interrupt disabled for this MB; 1: interrupt generated when IFLAG1 bit is set after TX/RX completion.

CAN_IFLAG1 — Interrupt Flag 1 Register

Bit(s)	Name	Reset	Description
31:0	`BUF31I–BUF0I`	0	Message buffer interrupt flags. Set by hardware when MB TX completes (TX CODE → INACTIVE) or RX frame arrives. Write 1 to clear. Must clear before re-arming MB for reception.

Message Buffer CS Word (CAN_MB[n].CS)

Bit(s)	Name	Reset	Description
31:16	`TIMESTAMP[15:0]`	—	Time stamp. Captured from the CAN free-running timer when the frame is received or transmitted (if TSYN/TTCAN enabled).
15:12	`CODE[3:0]`	—	Mailbox code. RX: 0x0=inactive; 0x4=empty (ready to receive); 0x2=full (holds received frame); 0x6=overrun. TX: 0x8=inactive; 0xC=ready to transmit (triggers TX); 0x9=abort. Set CODE=0x4 to arm MB for RX; set CODE=0xC to trigger TX.
11	`SRR`	0	Substitute remote request (extended frames only). Must be 1 for extended frames transmitted as substitute for RTR in the base ID field.
10	`IDE`	0	ID extended bit. 0: standard 11-bit ID; 1: extended 29-bit ID.
9	`RTR`	0	Remote transmission request. 0: data frame; 1: remote frame.

3:0	**DLC[3:0]**	0	Data length code. 0–8 for classic CAN. CAN FD: 0–15 (see DLC-to-byte mapping table).

4.3 SDK Usage Example

```
#include "flexcan_driver.h"
flexcan_state_t state;
const flexcan_user_config_t cfg={
    .bitrate={.propSeg=7,.phaseSeg1=4,.phaseSeg2=1,.preDivider=4,.rJumpwidth=1},
    .max_num_mb=16U,.is_rx_fifo_needed=false,.fd_enable=false,
    .payload=FLEXCAN_PAYLOAD_SIZE_8,
};
FLEXCAN_DRV_Init(INST_CANCOM1,&state,&cfg);
/* TX */
flexcan_data_info_t ti={.msg_id_type=FLEXCAN_MSG_ID_STD,.data_length=8U};
uint8_t txd[8]={1,2,3,4,5,6,7,8};
FLEXCAN_DRV_Send(INST_CANCOM1,1U,&ti,0x100U,txd,100U);
/* RX */
flexcan_data_info_t ri={.msg_id_type=FLEXCAN_MSG_ID_STD,.data_length=8U};
FLEXCAN_DRV_ConfigRxMb(INST_CANCOM1,2U,&ri,0x200U);
FLEXCAN_DRV_SetRxIndividualMask(INST_CANCOM1,FLEXCAN_MSG_ID_STD,2U,0x7FFU);
flexcan_msgbuff_t rxb;
FLEXCAN_DRV_Receive(INST_CANCOM1,2U,&rxb);
```

5. CAN in AUTOSAR

5.1 ComStack Architecture

Layer	Module	Responsibility
Application	RTE	Generates/consumes signals; calls COM APIs.
Service	ComM	Manages network communication mode (FULL_COM, SILENT_COM, NO_COM).
Service	PduR	Routes PDUs between COM, CanTp, CanIf, Dcm.
Service	COM	Signal packing/unpacking, I-PDU composition, signal gateway, timeout monitoring.
Service	CanTp	ISO 15765-2 segmentation and reassembly for multi-frame messages (UDS, OBD).
Service	CanNm	CAN network management — sleep/wake coordination (AUTOSAR NM algorithm).
MCAL	CanIf	Abstracts physical channel; PDU-level TX/RX; acceptance filtering by PDU ID.
MCAL	Can	Hardware register access; mailbox management; ISR routing to CanIf.
MCAL	CanTrcv	Controls transceiver mode: Normal, Standby, Sleep, Wakeup.

5.2 Can Driver Configuration — Full Field Definitions

Config Field	Type / Values	Full Description
CanControllerId	uint8	Unique identifier for this CAN controller instance. Referenced by all upper-layer modules via CanIf.
CanControllerBaseAddress	uint32	Base address of the CAN hardware peripheral (e.g., CAN0_BASE). Used by driver to map registers.
CanControllerBaudRate	uint16 (kbit/s)	Nominal baud rate in kbit/s. Driver selects prescaler/segment values to achieve this rate from the peripheral clock source.
CanControllerPropSeg	uint8 (1–8 TQ)	Propagation segment length in time quanta. Compensates for physical bus propagation delay including cable and transceiver.

CanControllerSeg1	uint8 (1–16 TQ)	Phase buffer segment 1 in TQ. Combined with PropSeg defines the sample point position. Can be lengthened during resynchronisation.
CanControllerSeg2	uint8 (2–8 TQ)	Phase buffer segment 2 in TQ. Duration after sample point. Must be ≥ SJW. Can be shortened during resync.
CanControllerSyncJumpWidth	uint8 (1–4 TQ)	Synchronisation jump width. Maximum number of TQ the bit time can be adjusted per resynchronisation event.
CanDefaultBaudrateConfigID	uint8	Index into the CanControllerBaudrateConfig array selecting the default baud rate at startup.

HOH Config Field	Type / Values	Full Description
CanObjectId	uint8	Hardware Object Handle identifier. Unique ID used in Can_Write() and CanIf to reference this TX or RX object.
CanObjectType	CAN_HOH_TX / CAN_HOH_RX	Defines whether this Hardware Object Handle is used for transmission or reception.
CanIdType	CAN_STANDARD / CAN_EXTENDED / CAN_MIXED	Type of CAN IDs handled: 11-bit, 29-bit, or both. MIXED allows both within the same HOH.
CanHwObjectCount	uint8	Number of hardware mailboxes or FIFO slots assigned to this HOH. Multiple TX slots allow pipelining.
CanFilterMaskRef	ptr to CanFilterMask	Reference to the filter mask applied to incoming messages for RX HOHs. NULL = accept all.
CanControllerRef	uint8	Reference to the CanControllerId that owns this HOH.

5.3 CanIf Configuration — Full Field Definitions

CanIf TX PDU Field	Type	Full Description
CanIfTxPduId	PduIdType	Unique PDU identifier within CanIf. Used in CanIf_Transmit() call from PduR.
CanIfTxPduCanId	uint32	CAN ID of this PDU. For standard frames: bits [10:0]. For extended: bits [28:0]. Bit 31 set for extended in some implementations.

CanIfTxPduCanIdMask	uint32	Mask applied to CanIfTxPduCanId to allow range-based matching or multiplexed PDUs. 0x7FF = exact match for 11-bit IDs.
CanIfTxPduDlc	uint8	Data length code. Number of bytes expected in the payload (0–8 classic, 0–64 FD).
CanIfTxPduType	STATIC / DYNAMIC	STATIC: CAN ID fixed at configuration time. DYNAMIC: CAN ID set at runtime via CanIf_SetDynamicTxId().
CanIfTxPduHohIdRef	CanObjectId	Reference to the TX Hardware Object Handle that will carry this PDU.
CanIfTxPduUserTxConfirmation	fn ptr	Callback function (e.g., PduR_CanIfTxConfirmation) invoked by CanIf after successful TX. Must not block.
CanIfTxPduTriggerTransmit	fn ptr	Optional callback invoked by CanIf to request TX data from upper layer just before transmission (for time-triggered or on-demand PDUs).

CanIf RX PDU Field	Type	Full Description
CanIfRxPduId	PduIdType	Unique PDU identifier for reception. Passed to the upper-layer RxIndication callback.
CanIfRxPduCanId	uint32	CAN ID to match for this PDU. Must match (after mask application) the received frame's CAN ID.
CanIfRxPduCanIdMask	uint32	Mask applied to received CAN ID before comparison: (received_id & mask) == (CanIfRxPduCanId & mask). 0x7FF = exact match.
CanIfRxPduDlc	uint8	Expected data length. If DLC check is enabled and received DLC < configured DLC, frame is rejected.
CanIfRxPduHohIdRef	CanObjectId	Reference to the RX HOH this PDU is received on.
CanIfRxPduUserRxIndication	fn ptr	Callback (e.g., PduR_CanIfRxIndication) invoked when a matching frame arrives. PDU data is passed via PduInfoType.
CanIfRxPduReadData	bool	TRUE: received data is stored in a shadow buffer accessible via CanIf_ReadRxPduData(); FALSE: data only delivered via callback.
CanIfRxPduReadNotifyStatus	bool	TRUE: enables CanIf_GetRxPduStatus() to report last RX result; used for polling-based upper layers.

5.4 CanTp (ISO 15765-2) Frame Type Definitions

Frame Type	N_PCI Byte	Description of All Fields

Single Frame (SF)	N_PCI[7:4]=0x0	N_PCI[3:0]=SF_DL (0–7 for classic, 0 then second byte for CAN FD). Entire message fits in one frame. Upper layer receives payload immediately.
First Frame (FF)	N_PCI[7:4]=0x1	N_PCI[3:0]+byte2 = FF_DL (total message length, 12-bit for classic). CanTp sends Flow Control request to sender; upper layer notified after last CF arrives.
Consecutive Frame (CF)	N_PCI[7:4]=0x2	N_PCI[3:0]=SN (Sequence Number 0–15, wraps). Carries up to 7 bytes of payload. Must arrive within N_Cr timeout.
Flow Control (FC)	N_PCI[7:4]=0x3	N_PCI[3:0]=FS (Flow Status: 0=ContinueToSend, 1=Wait, 2=Overflow). Byte1=BS (Block Size, 0=no limit). Byte2=STmin (min separation time 0–127 ms or 0xF1–0xF9 = 100–900 µs).

6. CAN on Kinetis KV5x (FlexCAN)

6.1 Hardware Overview

KV5x FlexCAN: 16 Message Buffers, Rx FIFO, individual and global masks, configurable PCS (up to 6 lines), legacy CAN 2.0A/B only (no CAN FD). Peripheral clock selectable: bus clock or oscillator.

6.2 Complete Register Bit Definitions

CAN_MCR — Module Configuration Register (KV5x)

Bit(s)	Name	Reset	Description
31	`MDIS`	0	Module disable. 0: FlexCAN enabled; 1: module clock gated off for low power.
30	`FRZ`	0	Freeze enable. 0: normal; 1: enables freeze mode when HALT is asserted. Required to access configuration registers after first enable.
29	`FEN`	0	FIFO enable (Rx FIFO). 0: Rx FIFO disabled; 1: Rx FIFO enabled using first 8 MBs. Legacy name; same as RFEN in S32K.
28	`HALT`	0	Halt FlexCAN. Write 1 to enter freeze/halt mode. Confirmed by FRZACK. Write 0 to resume.
27	`NOTRDY`	—	Not ready (read-only). 1: FlexCAN in low-power mode or initialising.
26	`WAKMSK`	0	Wake-up interrupt mask. 1: interrupt on self-wake-up.
25	`SOFTRST`	0	Software reset. Write 1 to reset internal logic (not config regs). Self-clearing.
24	`FRZACK`	—	Freeze acknowledge (read-only). 1: CAN is in freeze mode; register writes permitted.
22	`SUPV`	1	Supervisor mode. 1: privileged access required for certain registers.
21	`SLFWAK`	0	Self wake-up. 1: FlexCAN wakes from stop mode on bus activity.
20	`WRNEN`	0	Warning interrupt enable. 1: TX/RX warning count interrupts enabled.
17	`LPDIS`	0	Low-power mode disable. 0: low-power mode allowed; 1: prevented.
16	`IRMQ`	0	Individual Rx mask and queue enable. 0: global mask; 1: per-MB RXIMR array. Note: on KV5x, MB14 and MB15 always use CAN_RX14MASK and CAN_RX15MASK regardless of IRMQ.

14:13	`IDAM[1:0]`	0	ID acceptance mode for Rx FIFO. 00: one full ID; 01: two 16-bit IDs; 10: four 8-bit fragments; 11: filter disabled (accept all).
6:0	`MAXMB[6:0]`	0x0F	Maximum MB index (0-based). Active MBs = MAXMB+1. Must be configured before clearing HALT.

CAN_CTRL1 — Control 1 Register (KV5x)

Bit(s)	Name	Reset	Description
31:24	`PRESDIV[7:0]`	0	Prescaler. CAN clock = bus/oscillator clock / (PRESDIV+1). Sets TQ period.
23:22	`RJW[1:0]`	0	Resync jump width minus 1. 0=1TQ, 1=2TQ, 2=3TQ, 3=4TQ.
21:19	`PSEG1[2:0]`	0	Phase segment 1 minus 1. 1–8 TQ. Can be lengthened during resync.
18:16	`PSEG2[2:0]`	0	Phase segment 2 minus 1. 2–8 TQ (minimum 2). Can be shortened during resync.
15	`BOFFMSK`	0	Bus-off interrupt mask. 1: interrupt when BOFFINT set.
14	`ERRMSK`	0	Error interrupt mask. 1: interrupt when ERRINT set.
13	`CLKSRC`	0	Clock source. 0: oscillator clock; 1: bus clock. Must match system clock configuration.
12	`LPB`	0	Loopback enable. 1: TX internally looped to RX for self-test.
11	`TWRNMSK`	0	TX warning interrupt mask. 1: interrupt when TEC reaches warning threshold.
10	`RWRNMSK`	0	RX warning interrupt mask. 1: interrupt when REC reaches warning threshold.
7	`SMP`	0	Sampling mode. 0: single sample; 1: triple sample with majority vote.
6	`BOFFREC`	0	Bus-off recovery. 0: automatic; 1: manual (software must request re-entry into initialisation mode).
5	`TSYN`	0	Timer sync. 1: free-running timer resets on received SOF.
4	`LBUF`	0	Lowest buffer first. 0: lowest ID priority; 1: lowest index first.
3	`LOM`	0	Listen-only mode. 1: receive only, no ACK/error frames transmitted.
2:0	`PROPSEG[2:0]`	0	Propagation segment minus 1. 1–8 TQ. Compensates bus signal delay.

CAN_ESR1 — Error and Status 1 Register (KV5x)

Bit(s)	Name	Reset	Description
19	`SYNCH`	—	Bus synchronised (read-only). 1: node is synchronised and can communicate.
18	`TWRNINT`	0	TX warning interrupt flag. TEC reached warning limit. Write 1 to clear.
17	`RWRNINT`	0	RX warning interrupt flag. REC reached warning limit. Write 1 to clear.
16	`BIT1ERR`	0	Bit 1 error: transmitted recessive, monitored dominant. Write 1 to clear.
15	`BIT0ERR`	0	Bit 0 error: transmitted dominant, monitored recessive. Write 1 to clear.
14	`ACKERR`	0	ACK error: no dominant bit in ACK slot. Write 1 to clear.
13	`CRCERR`	0	CRC mismatch between received and calculated values. Write 1 to clear.
12	`FRMERR`	0	Form error: illegal bit in fixed-format field. Write 1 to clear.
11	`STFERR`	0	Stuff error: > 5 consecutive equal bits. Write 1 to clear.
10	`TXWRN`	—	TEC ≥ warning threshold (read-only).
9	`RXWRN`	—	REC ≥ warning threshold (read-only).
8	`IDLE`	—	Bus idle: 11+ consecutive recessive bits detected.
7	`TX`	—	FlexCAN transmitting (read-only).
6:5	`FLTCONF[1:0]`	—	Fault confinement. 00=error active; 01=error passive; 1x=bus off.
4	`RX`	—	FlexCAN receiving (read-only).
2	`BOFFINT`	0	Bus-off interrupt flag. Write 1 to clear.
1	`ERRINT`	0	Error interrupt flag. Write 1 to clear.
0	`WAKINT`	0	Wake-up interrupt flag. Write 1 to clear.

CAN_RXMGMASK — Rx Global Mask (KV5x)

Bit(s)	Name	Reset	Description
31:0	`MG[28:0] in bits 31:3`	0xFFFFFFFF	Global receive mask applied to all MBs when IRMQ=0 (or to MBs other than MB14/15 when IRMQ=1). Bit=1: must match filter ID bit; Bit=0: don't care. Extended IDs: bits [31:3]; Standard IDs: bits [31:21].

CAN_RX14MASK / CAN_RX15MASK — Individual MB14/15 Masks (KV5x)

Bit(s)	Name	Reset	Description
31:0	`RX14M / RX15M`	0xFFFFFFFF	Individual receive masks for MB14 and MB15 respectively. Same bit format as RXMGMASK. Always used for MB14/15 regardless of IRMQ setting. Allows two MBs to have unique filter masks independent of global configuration.

CAN_ECR — Error Counters Register (KV5x)

Bit(s)	Name	Reset	Description
15:8	`TXERRCNT[7:0]`	0	Transmit error counter. Incremented on each TX error (by 8 for some error types), decremented on successful TX. ≥128 = error passive; ≥256 = bus off.
7:0	`RXERRCNT[7:0]`	0	Receive error counter. Incremented on RX errors, decremented on successful reception. ≥128 = error passive.

6.3 MCUXpresso SDK Example

```
#include "fsl_flexcan.h"
flexcan_config_t cfg;
FLEXCAN_GetDefaultConfig(&cfg);
cfg.baudRate=500000U;
cfg.enableIndividMask=true;
FLEXCAN_Init(CAN0,&cfg,CLOCK_GetFreq(kCLOCK_BusClk));
/* RX MB0: accept 0x123 */
FLEXCAN_SetRxIndividualMask(CAN0,kFLEXCAN_FrameFormatStandard,0,0x7FFU);
flexcan_rx_mb_config_t rmc={.format=kFLEXCAN_FrameFormatStandard,
    .type=kFLEXCAN_FrameTypeData,.id=FLEXCAN_ID_STD(0x123)};
FLEXCAN_SetRxMbConfig(CAN0,0,&rmc,true);
/* TX MB1 */
FLEXCAN_SetTxMbConfig(CAN0,1,true);
flexcan_frame_t tf={.format=kFLEXCAN_FrameFormatStandard,
    .type=kFLEXCAN_FrameTypeData,.id=FLEXCAN_ID_STD(0x456),.length=8};
tf.dataByte0=0xDE; tf.dataByte1=0xAD;
FLEXCAN_TransferSendBlocking(CAN0,1,&tf);
```

7. Platform Comparison & Best Practices

7.1 Feature Comparison

Feature	STM32F4	ESP32	S32K1	AUTOSAR	KV5x
CAN Std	2.0A/B	2.0B	2.0A/B+FD*	Config	2.0A/B
TX Objects	3 mailboxes	1+queue	32–64 MBs	Config	16 MBs
RX FIFO	2×3 msg	64 bytes	FIFO+MBs	Config	FIFO+MBs
Per-MB Mask	No	N/A	Yes (IRMQ)	CanIf	MB14/15 only
CAN FD	No	No	K14x/K148	Config	No
DMA	No	No	K14x+	Yes	No
Bus-Off Auto	ABOM bit	API call	BOFFREC	Config	BOFFREC
Listen-Only	SILM bit	LOM mode	LOM bit	N/A	LOM bit

* CAN FD on S32K variants K14x and K148 only.

7.2 Common Pitfalls

Symptom	Root Cause	Fix
No ACK, immediate error passive	No termination or only one node	Add 120Ω at both ends; use loopback for single-node test
Bus Off immediately	Bit timing mismatch	Verify all nodes use same baud rate; scope to measure actual TQ
CRC/form errors at high speed	Cable too long or poor termination	Reduce speed or shorten cable; verify 120Ω at both ends
STM32 filter not matching	Wrong ID bit position	Standard ID: shift left 5 bits in filter register; Extended: shift left 3
S32K/KV5x MB stuck FULL	CODE not reset to 0x4	After reading RX MB, write CODE=0x4 to re-arm
ESP32 stops after bus error	TWAI_ALERT_BUS_OFF not handled	Read alerts, call twai_initiate_recovery() then twai_start()
AUTOSAR Seq never completes	ISR not bound to Can_Isr	Verify OS ISR table links CAN vector to CanIsr routine

8. Appendix

8.1 CAN FD DLC Encoding

DLC	Classic Bytes	CAN FD Bytes
0–8	0–8	0–8
9	N/A	12
10	N/A	16
11	N/A	20
12	N/A	24
13	N/A	32
14	N/A	48
15	N/A	64

8.2 Common CAN ID Assignments

ID Range	Protocol / Usage
0x000	Highest priority — emergency/safety
0x001–0x07F	CANopen NMT, heartbeat
0x7DF	OBD-II functional request
0x7E0–0x7E7	UDS physical request (tester → ECU)
0x7E8–0x7EF	UDS response (ECU → tester)
0x7FF	Lowest priority standard ID

8.3 References

- STM32F4xx Reference Manual RM0090 — STMicroelectronics
- ESP-IDF Programming Guide: TWAI Driver — Espressif
- S32K1xx Reference Manual S32K1XXRM — NXP Semiconductors
- AUTOSAR_SWS_CANDriver.pdf, AUTOSAR_SWS_CANInterface.pdf, AUTOSAR_SWS_CANTransportLayer.pdf
- KV5x Reference Manual KV5XP144M240RM — NXP Semiconductors
- ISO 11898-1:2015, ISO 15765-2:2016, Bosch CAN Specification 2.0